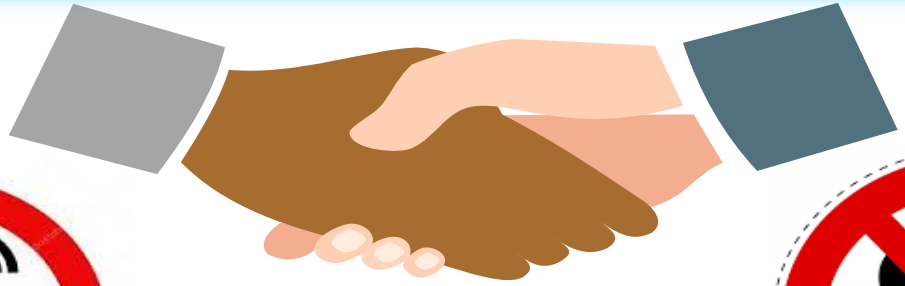
# SOYEZ LES BIENVENUES AU COURS DE PROGRAMMATION WEB 1 (JAVASCRIPT)



# Plan

# Références bibliographiques

- Cours du Professeur Bart Kimps
- Site du zéro tutoriel de Mathieu Nebra
- Tutoriel du site W3docs
  - https://fr.w3docs.com/apprendre-css.html
- W3 schools qui n'a aucun lien avec W3C
  - W3 School https://www.w3schools.com/
- Tutoriel du site de la console
  - https://laconsole.dev/formations/javascript/

# Javascript

# Amuse-toi bien?

Une dernière chose à ne pas oublier:

amuse-toi!

La programmation peut être une activité amusante et créative. Tout comme dessiner ou jouer. Une fois que tu auras compris comment coder, ta seule limite est ton  imagination.

Bienvenue dans le monde fantastique  de la programmation. J'espère que tu vas bien?

# Fonctions et local Storage

# Introduction

En JavaScript, les fonctions et le localStorage sont des outils très importants pour développer des applications web. Les **fonctions** permettent d'organiser et de réutiliser le code, tandis que le **localStorage** sert à stocker des données dans le navigateur. Leur utilisation facilite la création de pages web dynamiques et interactives.

# Fonctions

Les **fonctions** sont l'un des piliers de la programmation.

À la fin de ce chapitre, l'étudiant sera capable de:

| | |
|---|---|
| **Réutiliser le code** → | **éviter de répéter plusieurs fois les mêmes instructions** |
| Organiser le programme → | rendre ton code plus clair et structuré |
| Gagner du temps → | une fonction écrite = utilisée plusieurs fois |
| Faciliter la maintenance → | modifier une seule fonction au lieu de tout changer |
| Créer des programmes dynamiques → | passer des paramètres (valeurs) pour obtenir différents résultats |

Par définition, une fonction est un bloc de code réutilisable qui permet d'exécuter une tâche précise. Elle évite de répéter plusieurs fois le même code.

# 1. Syntaxe d’une fonction(1/2)

Une fonction en JavaScript est définie en utilisant le mot-clé function, suivi d'un nom, d'une liste de paramètres entre parenthèses et d'un bloc de code entre accolades.

# 2. Syntaxe d'une fonction(2/2)

```
function nomDeLaFonction() {
  // instructions
}
```

# 3. Exemple

```
function direBonjour() {
console.log("Bonjour !");
}
```

Pour exécuter une fonction, il suffit de l'appeler par son nom suivi de parenthèses.

Dans notre cas: `direBonjour(); // Bonjour !`

# 4. Les fonctions avec paramètres

Les **paramètres** permettent de passer des valeurs à la fonction.

```
function saluer(nom) {
console.log("Bonjour " + nom);}
saluer("Ali"); //Bonjour Ali
saluer("Marie"); //Bonjour Marie
```

# 5. Les fonctions avec return

Une fonction peut renvoyer une valeur grâce à return.

```
function addition(a, b) {  return a + b;}
let resultat = addition(5, 3);
console.log(resultat); // 8
```

# 6. Différence entre afficher & retourner

```
// Affiche seulement

Function test1() {  console.log(10);}


// Retourne une valeur

function test2() {  return 10;}
```

# 7. Remarque:

return est plus puissant car on peut réutiliser la valeur.

# 8. Les types de fonctions

En JavaScript, il existe plusieurs types de fonctions, chacun ayant ses propres caractéristiques et usages, permettant ainsi d'écrire un code plus flexible, lisible et adapté aux différents besoins de programmation.

# a. Fonction classique

La méthode classique bénéficie du "hoisting", ce qui signifie qu'elle peut être appelée avant même d'être définie dans le code.

```
function carre(x) {  return x * x;}

function saluer(nom) {  return "Bonjour " + nom;}
```

# b.Expression de fonction(function expression)

Ici, on stocke la fonction dans une variable.

Elle n'est pas "hoistée".

```
const direAuRevoir = function(nom) {
return "Au revoir " + nom;};
```

# c. fonction fléchée (Arrow Function)

Introduite en ES6, elle est plus courte et ne possède pas son propre contexte this (très utile pour les rappels/callbacks).

```
let add = (a, b) => a + b;
console.log(add(2, 3)); //  5
const carre = (n) => n * n;
```

# Exemple

```
let person = {    name: 'Alice',
actions: ['walk', 'talk', 'eat’],
   logActions() {
this.actions.forEach(action => {
console.log(this.name + " can " + action);
});    }};person.logActions();
```

# d. Fonction anonyme

C'est une fonction qui n'a pas de nom. On l'utilise souvent comme argument d'une autre fonction.

```
setTimeout(function() {
console.log("Temps écoulé !");}, 1000);
```

# 9. Fonctions natives(1/4)

Comme tout langage de programmation, JavaScript ne déroge pas à la règle et vient avec son lot de fonction natives. Il suffit alors de **les appeler par leur nom**.

```
let age = parseInt("43");
```

# 9. Fonctions natives(2/4)

parseInt() permet de convertir une chaine de caractères en nombre entier. Si certaines fonctions comme parseInt() sont dites globales(peuvent s'exécuter de « partout » dans le code), d'autres vont être associées à un contexte objet(un prototype), **on parle de méthodes**.

# 10. Fonctions natives(3/4)

window.alert("Hello !"); // Affiche un pop-up textuelle

console.log("Hello !"); // Affiche un message dans la console

Math.random(); // Renvoie un nombre flottant aléatoire compris dans l'intervalle [0, 1[

# 11. Fonctions natives(4/4)

Il existe de nombreuses autres fonctions comme prompt(), confirm(), floor…

# 12. Fonctions de manipulation de données (les plus utilisées)(1/2)

Le JavaScript propose des fonctions natives (méthodes) pour transformer vos données, notamment pour les tableaux (**Arrays**).

# 13. Fonctions de manipulation de données (les plus utilisées)

| map() | Crée un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine. |
|---|---|
| **filter()** | Crée un nouveau tableau avec tous les éléments qui passent un test spécifique |
| **forEach()** | Exécute une fonction donnée sur chaque élément du tableau(ne retourne rien) |
| **reduce()** | Réduit le tableau en une seule valeur(ex:faire la somme de tous les nombres). |
| **Find()** | Renvoie la valeur du **premier** élément qui satisfait une condition, |

# 14. Fonctions asynchrones

Avec l'évolution du Web, la gestion du temps est devenue cruciale.

- **async / await** : Permet d'écrire du code asynchrone qui ressemble à du code synchrone, rendant la lecture plus facile.
- **then() / catch()** : Utilisées avec les Promesses pour gérer le succès ou l'échec d'une opération.

# 15. Conclusion

**Les fonctions** sont essentielles en JavaScript. Elles permettent de structurer, simplifier et rendre le code plus efficace. Leur maîtrise est indispensable pour créer des applications web modernes.